E04 Hit Feedback

CRE132 worksheet

What you'll make

Three small reactions that make a hit feel solid: the whole game freezes for a split second (**hit stop**), the slime flashes pure white, and the damage floats up above it as a number and fades.

By the end you'll know how `Time.timeScale` pauses a game and why some timers must use real time, how a script can listen to another script's C# event, and how to put text in the game world with TextMeshPro.

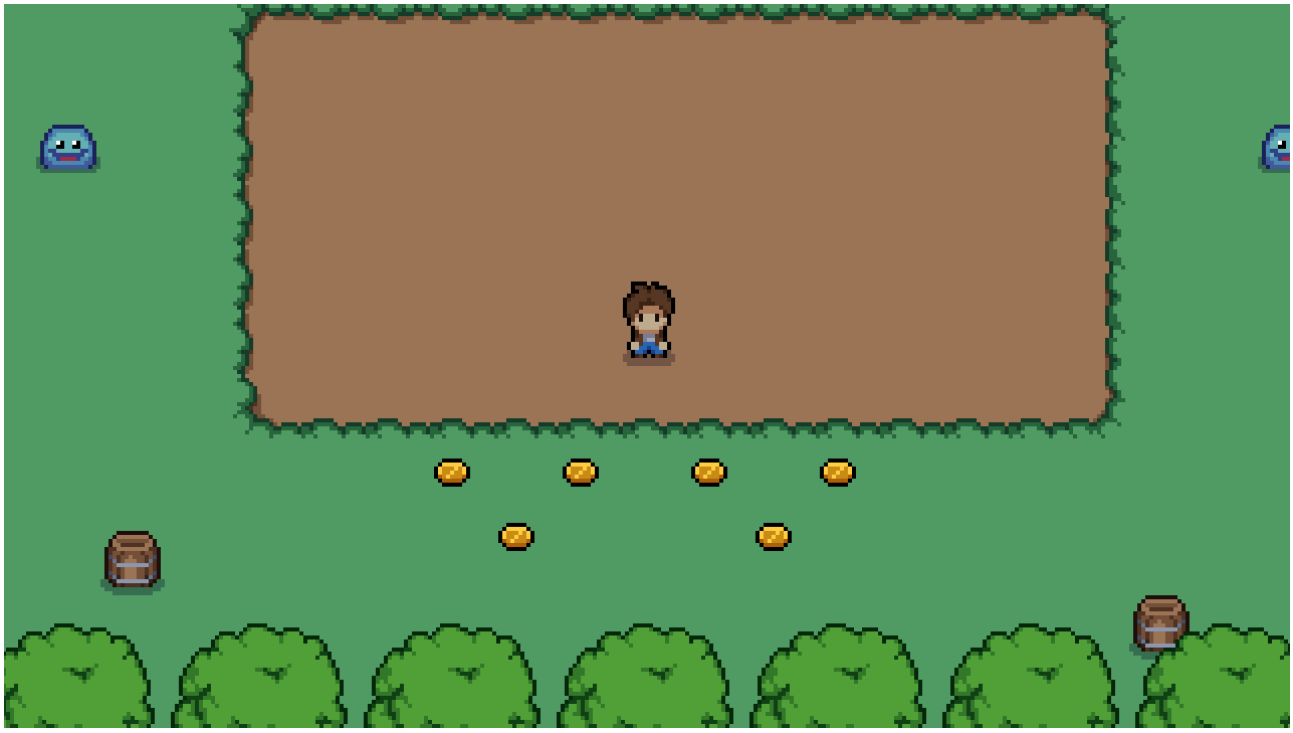


Figure 1: The finished scene: the player hitting a slime dummy, with a damage number above it.

Before you start

1. Unzip `CRE132_E04_HitFeedback_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_E04_HitFeedback_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Effects/E04_HitFeedback`.
3. Press **Play**. Walk with WASD or the arrows, swing the sword with **J** or the left mouse button, and throw a fireball with the right mouse button or **K**. The slimes have 10 health and turn redder as they're hurt, but a hit has no punch yet.

The scene has the **Player**, five **Slime Dummies**, three **Barrels** and an empty object called **Hit Stop**. The white flash needs a special shader, which is provided: the slimes already use the **Sprite Flash** material (`Assets/_game/Materials/Effects`). You'll make the scripts and a **Damage Number** prefab.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Freezing the game: HitStop

Make a folder `Assets/_game/Scripts/E04_HitFeedback` and create a script in it called `HitStop`:

```
// Freezes the whole game for a split second ("hit stop"), which makes hits feel heavy.
// It lives on its own object (not on the thing that was hit), so the game always
// unfreezes, even if the thing that was hit is destroyed during the freeze.
using System.Collections;
using UnityEngine;

public class HitStop : MonoBehaviour
{
    private bool isFrozen;

    public void Freeze(float seconds)
    {
        if (!isFrozen)
        {
            StartCoroutine(FreezeRoutine(seconds));
        }
    }

    private IEnumerator FreezeRoutine(float seconds)
    {
        isFrozen = true;
        Time.timeScale = 0f;
        // Realtime: normal WaitForSeconds uses scaled time, which has just stopped.
        yield return new WaitForSecondsRealtime(seconds);
        Time.timeScale = 1f;
        isFrozen = false;
    }
}
```

- **`Time.timeScale`** is how fast game time runs: 1 is normal, 0 stops it. Physics, animation and anything using `Time.deltaTime` all stop.
- **`WaitForSecondsRealtime`** counts real seconds. Plain `WaitForSeconds` counts game seconds, and with the time scale at 0 it would wait for ever.
- **`isFrozen`** stops two hits in the same moment from starting two freezes.
- **Why its own object?** A coroutine stops when its object is destroyed. If the freeze ran on the slime and that hit killed it, the game would stay frozen.

Add it to the **Hit Stop** object. There's nothing to set.

2. Damage numbers: DamageNumber

Create `DamageNumber` in the `E04_HitFeedback` folder:

```
// A number that pops up over something that was hit, floats upwards and fades out.
using System.Collections;
using TMPro;
using UnityEngine;

public class DamageNumber : MonoBehaviour
{
    [SerializeField] private float riseDistance = 0.8f;
    [SerializeField] private float duration = 0.7f;

    public void Show(int amount)
    {
```

```
        GetComponent<TMP_Text>().text = amount.ToString();
        StartCoroutine(Rise());
    }

    private IEnumerator Rise()
    {
        TMP_Text text = GetComponent<TMP_Text>();
        Vector3 start = transform.position;
        // Start a little to the side at random, so several numbers don't sit on top of
        // each other.
        start.x = start.x + Random.Range(-0.3f, 0.3f);
        float time = 0f;

        while (time < duration)
        {
            time = time + Time.deltaTime;
            float progress = time / duration;
            transform.position = start + new Vector3(0f, riseDistance * progress, 0f);
            text.alpha = 1f - progress * progress;
            yield return null;
        }

        Destroy(gameObject);
    }
}
```

- **`using TMPro`** gives access to TextMeshPro. `TMP_Text` is the text component; setting `.text` changes what it says.
- **The `while` loop with `yield return null`** runs once a frame. `progress` goes from 0 to 1 over `duration` seconds and moves the number up and fades it out.
- **`1f - progress * progress`** fades slowly at first and quickly at the end, so you can read the number before it goes.

Make the prefab: choose **GameObject > 3D Object > Text - TextMeshPro** and rename it **Damage Number**. Set:

Component	Setting	Value
Rect Transform	**Pos X, Y, Z**	0, 0, 0
	Width, **Height**	2, 1
TextMeshPro - Text	**Text Input**	1
	Font Size	5
	Font Style	**B** (bold)
	Vertex Color	pale yellow, hex FFF299
	Alignment	centre and middle
Mesh Renderer	**Sorting Layer**	Foreground

Add the **DamageNumber** component (leave **Rise Distance** 0.8 and **Duration** 0.7). Make a folder `Assets/_game/Prefabs/Effects`, drag **Damage Number** into it to make a prefab, and delete it from the Hierarchy.

- **A 3D TextMeshPro** lives in the game world, like a sprite, so it can sit above a slime. The UI version would need a Canvas.
- **Foreground** draws it on top of the characters.

3. Reacting to hits: HitFeedback

This script ties it all together. Create `HitFeedback` in the `E04_HitFeedback` folder:

```
// Makes a hit on this character feel like a hit: the game freezes for a moment
// (HitStop), the sprite flashes white (the CRE132/Sprite Flash shader's Flash Amount),
// and the damage floats up as a number.
// It listens to Health's HealthChanged event (S09), so it works whatever did the damage.
using System.Collections;
using UnityEngine;

public class HitFeedback : MonoBehaviour
{
    [SerializeField] private float hitStopTime = 0.06f;
    [SerializeField] private float flashTime = 0.1f;
    [SerializeField] private DamageNumber damageNumberPrefab;

    private Health health;
    private SpriteRenderer spriteRenderer;
    private HitStop hitStop;
    private int lastHealth;

    private void Awake()
    {
        health = GetComponent<Health>();
        spriteRenderer = GetComponentInChildren<SpriteRenderer>();
        hitStop = FindAnyObjectByType<HitStop>();
    }

    private void Start()
    {
        lastHealth = health.CurrentHealth;
    }

    private void OnEnable()
    {
        health.HealthChanged += OnHealthChanged;
    }

    private void OnDisable()
    {
        health.HealthChanged -= OnHealthChanged;
    }

    private void OnHealthChanged(int currentHealth, int maxHealth)
    {
        int damage = lastHealth - currentHealth;
        lastHealth = currentHealth;
        // Healing changes health too; only react to damage.
        if (damage <= 0)
        {
            return;
        }

        hitStop.Freeze(hitStopTime);
        StartCoroutine(Flash());

        DamageNumber number = Instantiate(damageNumberPrefab,
            transform.position + new Vector3(0f, 1f, 0f), Quaternion.identity);
        number.Show(damage);
```

```
    }

    private IEnumerator Flash()
    {
        // .material gives this sprite its own copy of the material, so only this one
        // flashes.
        spriteRenderer.material.SetFloat("_FlashAmount", 1f);
        // Realtime, so the flash shows during the hit stop.
        yield return new WaitForSecondsRealtime(flashTime);
        spriteRenderer.material.SetFloat("_FlashAmount", 0f);
    }
}
```

- **`health.HealthChanged += OnHealthChanged`** subscribes: every time Health's number changes, Health calls our method. `-=` in `OnDisable` unsubscribes, so a removed slime doesn't leave a broken link behind.
- **Working out the damage:** Health tells us the new health, not the damage. Remembering `lastHealth` means `lastHealth - currentHealth` is how much was lost. Healing gives 0 or less, so we ignore it.
- **`FindAnyObjectByType<HitStop>()`** finds the one Hit Stop object in the scene, so you don't have to drag it into every slime.
- **The white flash:** tinting a sprite white does nothing (white times a colour is the same colour). The **Sprite Flash** shader has a **Flash Amount** that blends the sprite towards a solid colour: 1 is pure white, 0 is normal. `spriteRenderer.material` (not `sharedMaterial`) makes a copy for this slime, so the others don't flash too.

Add it: select all five **Slime Dummies** and add **HitFeedback**. Drag the **Damage Number** prefab into **Damage Number Prefab**. Leave **Hit Stop Time** at 0.06 and **Flash Time** at 0.1.

Test it: hit a slime. The game stops for a blink, the slime flashes white, and a **1** floats up and fades. Hit it quickly a few times: the numbers spread out sideways.

Check your work

- ☐ Every hit freezes the game for a moment, then it carries on normally.
- ☐ Only the slime you hit flashes white; the others don't.
- ☐ A yellow number appears above the slime, rises and fades, then disappears from the Hierarchy.
- ☐ The killing hit doesn't leave the game frozen.
- ☐ Compare with `CRE132_E04_HitFeedback_Finished` if something behaves differently.

Key points

- **Game feel is lots of tiny reactions.** 60 ms of freeze and 100 ms of flash make a hit feel solid; the player hardly notices them separately.
- **`Time.timeScale` stops nearly everything**, so anything that must keep going during the freeze (the unfreeze itself, the flash) uses real time.
- **C# events let scripts listen.** HitFeedback doesn't care what did the damage (sword, fireball, anything later), only that Health changed.

If something goes wrong

- **The game freezes for ever:** you used `WaitForSeconds` instead of `WaitForSecondsRealtime` in HitStop.
- **NullReferenceException at `hitStop.Freeze`:** there's no **HitStop** component in the scene. Add it to the **Hit Stop** object.

- **NullReferenceException at `Instantiate`, or UnassignedReferenceException:** the **Damage Number Prefab** slot is empty.
- **No number appears:** check the prefab's **Sorting Layer** is **Foreground**, and that its **Text Input** isn't empty.
- **The "TMPro" namespace could not be found:** check the `using TMPro;` line is spelled exactly.
- **No white flash:** the slime's **Sprite Renderer** must use the **Sprite Flash** material. If someone changed it, drag it back from `Materials/Effects`.

Try it

1. A bigger freeze for the hit that kills: in `OnHealthChanged`, use a longer time when `currentHealth` is 0.
2. Red numbers for big hits: give the fireball **Damage** 2, and in `DamageNumber.Show` set the text's `color` to red when `amount` is more than 1.
3. Hit stop and flash on the player when a slime hits them. (The Player would need **Health** and the Sprite Flash material; T08's `TopDownHurt` does the hurting.)
4. Change the **Flash Color** on the **Sprite Flash** material to red. Which feels better, white or red?